SCC.231 Coursework Report – Network Tester (Student: Bruce)

This report explains the work which I completed for the SCC.231 coursework. I was able to write a Mininet topology and a network tester program that works with both TCP and UDP. I will be explain each task in simple words. I have also added screenshots as further proof of the code and outputs of different section.

Task 1: Mininet Topology

In Task 1, using the template I created a custom Mininet topology. I added the all the hosts from 1 to5, both switches, and all 3 routers. I also set the IP addresses and routing rules. The topology was quite difficult to debug cause there was a lot of code prewritten so navigating everything in the VM was quite difficult when it came to finding certain problems, but the main structure of the Mininet topology now works. I will place screenshots of the topology test results below.

Starting Mininet

```
[sudo] mn --custom topology.py --topo courseworkTopo --switch lxbr --link tc
--controller nonebr --link tc --controller none
*** Adding controller
*** Adding hosts:
host1 host2 host3 host4 host5 router1 router2 router3
*** Adding switches:
switch1 switch2
*** Adding links:
(host1, switch1) (host2, switch1) (host3, switch2) (host4, switch2) (host5, router3) (10ms delay 1.00
000% loss) (10ms delay 1.00000% loss) (router1, router2) (100.00Mbit 100ms delay) *** Error: Warning:
 sch_htb: quantum of class 50001 is big. Consider r2q change.
(100.00Mbit 100ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q cha
nge.
(router1, router3) (switch1, router1) (switch2, router2)
*** Configuring hosts
host1 host2 host3 host4 host5 router1 router2 router3
*** Starting controller

*** Starting 2 switches
switch1 switch2
*** Starting CLI:
mininet>
```

interfaces on routers

```
mininet> router1 ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

router1-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.0.1  netmask 255.255.255.0  broadcast 192.168.0.255
        ether e6:0b:16:0e:1a:35  txqueuelen 1000  (Ethernet)
        RX packets 2  bytes 108 (108.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

router1-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.10.0.1  netmask 255.255.255.0  broadcast 10.10.0.255
        ether c6:d0:bc:de:57:2e  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

router1-eth3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.10.1.1  netmask 255.255.255.0  broadcast 10.10.1.255
        ether ae:84:27:4b:c3:1c  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

```
mininet> host1 ping -c 1 host2
PING 192.168.0.3 (192.168.0.3) 56(84) bytes of data.
64 bytes from 192.168.0.3: icmp_seq=1 ttl=64 time=0.109 ms

--- 192.168.0.3 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.109/0.109/0.109/0.000 ms
mininet> host1 ping -c 1 192.168.0.1
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=0.078 ms
```

Host ping in the same LAN

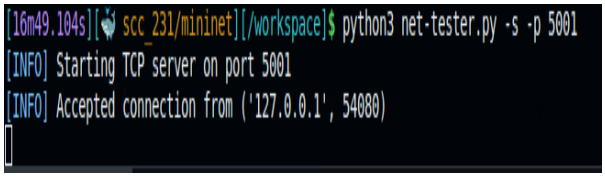
```
mininet> host1 ping -c 1 192.168.0.1
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=0.078 ms

--- 192.168.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.078/0.078/0.078/0.000 ms
mininet>
```

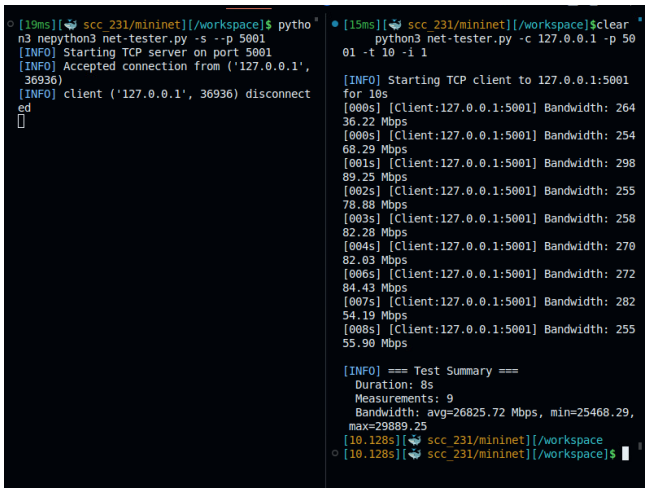
Host to router ping

Task 2: TCP Client and TCP Server

In Task 2, I was able to write a TCP server that accepts many clients. The TCP client sends data for a set amount of time, I used 10 seconds for testing. Every few seconds it prints the bandwidth. The Logger class shows the results. I tested this using localhost.



TCP Server running



TCP client starting + bandwidth output + test summary

Task 3: UDP Client (No ACK) and UDP Server

In Task 3, the UDP client are designed to send packets at a fixed rate. The server then is supposed measure the: bandwidth, jitter, and loss. This part was harder because UDP doesn't guarantee packet delivery. I chose to use packet IDs to calculate loss.

```
[15ms][🐳 python3 net-tester.py -s -u -p 5001
 -i 1t-tester.py -s -u -p 5001 -i 1
[INFO] Starting UDP server on port 5001 (ack=
False)
[000s] [Client:127.0.0.1:58002] Bandwidth: 0.
01 Mbps Loss: 0.00% Jitter: 0.000000 ms
[001s] [Client:127.0.0.1:58002] Bandwidth: 0.
98 Mbps Loss: 0.00% Jitter: 8.360211 ms
[002s] [Client:127.0.0.1:58002] Bandwidth: 0.
97 Mbps Loss: 0.00% Jitter: 8.415567 ms
[003s] [Client:127.0.0.1:58002] Bandwidth: 0.
98 Mbps Loss: 0.00% Jitter: 8.348129 ms
[004s] [Client:127.0.0.1:58002] Bandwidth: 0.
98 Mbps Loss: 0.00% Jitter: 8.340591 ms
[005s] [Client:127.0.0.1:58002] Bandwidth: 0.
98 Mbps Loss: 0.00% Jitter: 8.356770 ms
[006s] [Client:127.0.0.1:58002] Bandwidth: 0.
98 Mbps Loss: 0.00% Jitter: 8.359271 ms
[007s] [Client:127.0.0.1:58002] Bandwidth: 0.
98 Mbps Loss: 0.00% Jitter: 8.352000 ms
[008s] [Client:127.0.0.1:58002] Bandwidth: 0.
98 Mbps Loss: 0.00% Jitter: 8.352824 ms
[009s] [Client:127.0.0.1:58002] Bandwidth: 0.
98 Mbps Loss: 0.00% Jitter: 8.355071 ms
[INFO] client 127.0.0.1: 58002 finished sendi
ng
```

UDP Server running (ack=False)

```
[81ms][127][🐳 scc_231/mininet][/workspace]$
pyhton3 net-tespython3 net-tester.py -c 127.0
.0.1 -u -p 5001 -t 10 -i 1 -r 1000
[INFO] Starting UDP client to 127.0.0.1:5001
for 10s at 1000 Kbps (ack=False)
[000s] [Client:127.0.0.1:5001] Bandwidth: 0.9
9 Mbps
[001s] [Client:127.0.0.1:5001] Bandwidth: 0.9
7 Mbps
[002s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps
[003s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps
[004s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps
[005s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps
[006s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps
[007s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps
[008s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps

[INFO] === Test Summary ===
  Duration: 8s
  Measurements: 9
  Bandwidth: avg=0.98 Mbps, min=0.97, max=0.9
9
```

UDP Client Sending (ack=False) + Client Test Summary

Task 4: UDP With ACKs (Client + Server)

In Task 4 acknowledgements (ACKs) are added. So when the client sends a packet, the server sends one back with the ID and the timestamp. The client then uses this to calculate RTT(Round-Trip Time) and jitter. This task started to work after fixing some bugs and timing mistakes as well tuple assignment problems, which were also quite hard to debug due to the amount of lines of code that were written.

```
[18ms][🐳 scc_231/mininet][/workspace]$ pytho
001 -i 1
[INFO] Starting UDP server on port 5001 (ack=
True)
[000s] [Client:127.0.0.1:38582] Bandwidth: 0.
00 Mbps Loss: 0.00% Jitter: 26113.465309 ms
[001s] [Client:127.0.0.1:38582] Bandwidth: 0.
97 Mbps Loss: 1.61% Jitter: 8.218212 ms
[002s] [Client:127.0.0.1:38582] Bandwidth: 0.
98 Mbps Loss: 50.41% Jitter: 8.200772 ms
[003s] [Client:127.0.0.1:38582] Bandwidth: 0.
98 Mbps Loss: 66.67% Jitter: 8.192039 ms
[004s] [Client:127.0.0.1:38582] Bandwidth: 0.
98 Mbps Loss: 75.00% Jitter: 8.181527 ms
[005s] [Client:127.0.0.1:38582] Bandwidth: 0.
98 Mbps Loss: 80.13% Jitter: 8.202484 ms
[006s] [Client:127.0.0.1:38582] Bandwidth: 0.
97 Mbps Loss: 83.42% Jitter: 8.220616 ms
[007s] [Client:127.0.0.1:38582] Bandwidth: 0.
97 Mbps Loss: 85.78% Jitter: 8.207184 ms
[008s] [Client:127.0.0.1:38582] Bandwidth: 0.
98 Mbps Loss: 87.46% Jitter: 8.178176 ms
[009s] [Client:127.0.0.1:38582] Bandwidth: 0.
98 Mbps Loss: 88.86% Jitter: 8.191630 ms
[INFO] client 127.0.0.1:38582 done sending
```

UDP Server Running (ack=True)

```
[22ms][🐳 scc_231/mininet][/workspace]$ pytho
n3 net-tespython3 net-tester.py -c 127.0.0.1
-u -a -p 5001 -t 10 -i 1 -r 1000

[INFO] Starting UDP client to 127.0.0.1:5001
for 10s at 1000 Kbps (ack=True)
[000s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps Loss: 0.81% Jitter: 0.090814 ms
[001s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps Loss: 0.41% Jitter: 0.080699 ms
[002s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps Loss: 0.27% Jitter: 0.072602 ms
[003s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps Loss: 0.20% Jitter: 0.067292 ms
[004s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps Loss: 0.16% Jitter: 0.068161 ms
[005s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps Loss: 0.14% Jitter: 0.076792 ms
[006s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps Loss: 0.12% Jitter: 0.075050 ms
[007s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps Loss: 0.10% Jitter: 0.070315 ms
[008s] [Client:127.0.0.1:5001] Bandwidth: 0.9
8 Mbps Loss: 0.09% Jitter: 0.066642 ms

[INFO] === Test Summary ===
  Duration: 8s
  Measurements: 9
  Bandwidth: avg=0.98 Mbps, min=0.98, max=0.9
8
  Loss: avg=0.26%, min=0.09, max=0.81
  Jitter: avg=0.074263 ms, min=0.066642, max=
0.090814
```

UDP Client (ack=True) + Client Test Summary

Overall Summary

All tasks seem to be working after various tests as well as debugging. TCP bandwidth seems to report correctly. The UDP without ACK shows both jitter and loss. UDP with ACK shows the RTT, jitter, and better measurements. The Logger summary also shows everything at the end. This coursework helped me understand sockets, threading, and network measurements, it also gave me a deeper understanding in how routing works in a network.

End of Report